ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work entitled **"LoanGuard-AI: Loan Fraud Detection & Risk Analysis"** is submitted by

SOUMYADIP SINGHA (24B11AI402)
SUBHANKAR DAS (24B11AI406)
PRITAM SAHOO (24B11AI358)
RISHAV SINGH (24B11AI369)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2024-2028.

Project Guide
Dr. R. Suneetha,
Assistant Professor,
AIML

Head of the Department
Dr. K. Naga Bhargavi,
Associate Professor,
AIML

DECLARATION

We hereby declare that the Cornerstone project entitled **"LoanGuard-AI: Loan Fraud Detection & Risk Analysis"** is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date: 11-10-2026

By
SOUMYADIP SINGHA (24B11AI402)
SUBHANKAR DAS (24B11AI406)
PRITAM SAHOO (24B11AI358)
RISHAV SINGH (24B11AI369)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. R. Suneetha**, Assistant Professor, who has guided us throughout the project. The valuable support, encouragement, and guidance received have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. R. Suneetha**, Assistant Professor, **Dr. B. Manikyala Rao**, Assistant Professor and **Dr. D. Veeraswamy**, Assistant Professor, Department of AIML, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. K. Naga Bhargavi**, Associate Professor, for the inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh**, Dean-SOC, for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh**, Registrar, **Dr. A. Ramesh**, Pro Vice-Chancellor (Engineering & Sciences), **Dr. S. Rama Sree**, Pro Vice-Chancellor (Academics), **Dr. M.B. Srinivas**, Vice-Chancellor, **Dr. M. Sreenivasa Reddy**, Deputy Pro Chancellor and Management, Aditya University for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

In digital retail lending and commercial banking, fraudulent loan applications and severe credit default risks inflict multi-billion dollar charge-offs annually. Early detection of fraudulent applications is critical yet exceptionally challenging due to extreme class imbalance, where fraudulent events represent only a tiny fraction (2.05%) of total submissions. Traditional credit assessment models rely primarily on rigid threshold heuristics that fail to capture multidimensional interactions among financial indicators. This project presents **LoanGuard-AI**, an explainable machine learning system and decision-support architecture for automated loan fraud risk detection and underwriting analytics.

The system ingests and preprocesses a benchmark financial dataset of 50,000 loan applications containing four core financial attributes: loan amount requested, monthly income, credit/CIBIL score, and debt-to-income (DTI) ratio. Data validation and cleaning pipelines remove corrupted records and verify plausible economic ranges. To overcome severe class imbalance without introducing synthetic data leakage, the classification engine implements a class-weighted Logistic Regression model utilizing inverse class frequency weighting. Evaluated on a stratified 20% holdout test partition (10,000 records), the model achieves an overall Accuracy of 51.81%, Balanced Accuracy of 48.89%, Fraud Recall of 45.85%, Specificity of 51.93%, and Precision of 1.96%, yielding a net financial fraud mitigation savings of $2,583,800 across intercepted default exposures.

Decision threshold optimization (0.10 to 0.90) provides tunable sensitivity cutoffs, and transparent underwriting decision logic breaks down risk drivers into plain-English factor attributions. The production-ready system is deployed via an interactive Streamlit web application providing real-time risk scores (0%–100%), risk tier classifications, and actionable credit factor insights for underwriting professionals.

Keywords: Machine Learning, Loan Fraud Detection, Financial Risk Assessment, Imbalanced Classification, Logistic Regression, Cost-Sensitive Learning, Underwriting Analytics, Streamlit

LIST OF CONTENTS

S. No.	CONTENT	PAGE NO.
	LIST OF FIGURES	i
1	Chapter 1: Introduction	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
2	Chapter 2: System Overview	3
2.1	Existing System	3
2.2	Limitations of Existing System	3
2.3	Proposed System	3
2.4	Software Requirements	4
2.5	Hardware Requirements	4
3	Chapter 3: System Implementation	5
3.1	Overview of Development Tools	5
3.1.1	Development Environment: Python and Visual Studio Code	5
3.1.2	ML Library / Framework: scikit-learn, pandas, numpy and Streamlit	5
3.2	Dataset Description	6
3.3	Implementation Modules	7
3.3.1	Handling Data Inspection and Cleaning (data_cleaning.py)	7
3.3.2	Balanced Model Training and Serialization (train_model.py)	7
3.3.3	Comprehensive Diagnostic Metrics Pipeline (generate_reports.py)	8
3.3.4	Explainable Underwriting Rule Logic (app.py)	8
3.3.5	Real-Time Prediction and Risk Scoring in Streamlit (app.py)	8
3.4	Steps to Run the Project	8
4	Chapter 4: System Architecture	9
5	Chapter 5: Data Preprocessing	11
5.1	Introduction	11
5.2	Handling Missing Values and Data Integrity	11
5.3	Data Cleaning and Target Distribution	12
5.4	Outlier Detection and Range Validation	13
5.5	Feature Engineering and Variable Properties	13
5.6	Data Visualization	14
5.7	Train-Test Split	16
5.8	Summary	17
6	Chapter 6: Methodology and Algorithms	18

S. No.	CONTENT	PAGE NO.
6.1	Introduction	18
6.2	Classification Techniques	18
6.2.1	Logistic Regression with Balanced Class Weights	18
6.2.2	Threshold Calibration and Probability Modeling	19
6.2.3	Class Weighting vs Resampling for Imbalanced Data	19
6.3	Evaluation Metrics for Severe Class Imbalance	19
6.3.1	Confusion Matrix & Financial Risk Exposure	19
6.3.2	Balanced Accuracy, Precision, Recall & PR-AUC	20
6.4	Decision Threshold Tuning Methodology	20
6.5	Model Interpretability and Feature Importance	21
6.6	Model Evaluation Metrics Mathematical Definitions	22
7	Chapter 7: Results and Analysis	23
7.1	Introduction	23
7.2	Experimental Setup	23
7.3	Headline Classification Results & Metrics	23
7.4	Detailed Confusion Matrix Breakdown	24
7.5	Discrimination Analysis (ROC vs PR)	25
7.6	Threshold Optimization Analysis	26
7.7	Comprehensive Multi-Threshold Analysis Table	27
7.8	Model Calibration & Probability Reliability Analysis	28
7.9	Test Cases	29
7.10	Key Findings & Discussion	30
8	Chapter 8: Output Screens	31
9	Chapter 9: Conclusion	34
10	Chapter 10: Future Scope	35
11	Chapter 11: References	36

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig. 3.1	Development environment setup and project folder structure	5
Fig. 3.2	Installed library versions	6
Fig. 4.1	System architecture of the LoanGuard-AI System	9
Fig. 5.1	Missing values audit per feature before cleaning	11
Fig. 5.2	Class distribution of the target variable (fraud_flag)	12
Fig. 5.3	Box plots of all features after cleaning (outlier check)	13
Fig. 5.4	Output of data_cleaning.py showing dataset statistics and class distribution	14
Fig. 5.5	Feature distributions (univariate histograms with KDE)	15
Fig. 5.6	Feature correlation heatmap	16
Fig. 5.7	CIBIL score and DTI distributions by fraud outcome	16
Fig. 6.1	Output of train_model.py showing balanced training and evaluation	21
Fig. 6.2	Logistic Regression feature coefficients (odds ratio / risk drivers)	21
Fig. 6.3	Prediction probability distribution (Legitimate vs Fraud)	22
Fig. 6.4	Model calibration curve (reliability diagram)	22
Fig. 7.1	Confusion matrix (raw counts and normalized percentages)	24
Fig. 7.2	Financial cost and risk mitigation matrix	24
Fig. 7.3	Receiver Operating Characteristic (ROC) curve	25
Fig. 7.4	Precision-Recall curve with no-skill baseline	25
Fig. 7.5	Decision threshold trade-off curve across operating cutoffs	26
Fig. 7.6	Metrics comparison scorecard bar chart	26
Fig. 7.7	Financial savings and risk exposure across decision thresholds	27
Fig. 7.8	ROC and PR discrimination curves comparison	28
Fig. 7.9	Metrics comparison across evaluation criteria	28
Fig. 7.10	Model feature importance drivers bar chart	29
Fig. 7.11	Underwriting decision factor breakdown for sample loan application	29
Fig. 8.1	Starting the Streamlit web application	31
Fig. 8.2	Streamlit web app: Clear / Low-Risk application evaluation	31
Fig. 8.3	Streamlit web app: Flagged high fraud risk application with risk drivers	32
Fig. 8.4	Streamlit web app: Caution / Moderate risk application evaluation	32
Fig. 8.5	Automated diagnostic reporting execution terminal output	33
Fig. 8.6	Executive validation report markdown summary view	33

CHAPTER 1
INTRODUCTION

1.1 Introduction

In the contemporary global financial ecosystem, digital retail lending has experienced exponential growth, driven by the ubiquity of online banking, automated credit approval engines, and decentralized fintech platforms. While digitalization significantly reduces customer acquisition friction and shortens loan approval turnaround times, it simultaneously exposes financial institutions to sophisticated lending fraud, deliberate credit defaults, and organized syndicate attacks. Fraudulent loan applicants frequently manipulate financial records, misstate monthly income, conceal existing credit obligations, or leverage synthetic identities to extract capital without intent to repay.

According to industry risk benchmarks, unsecured lending fraud accounts for billions of dollars in annual charge-offs globally. Credit underwriters face a dual challenge: protecting institutions against catastrophic capital losses while avoiding excessive false alarms that burden operational review pipelines and alienate creditworthy borrowers. Machine learning (ML) classification systems provide a data-driven paradigm to address this dilemma. By discovering non-linear relationships and high-dimensional interactions across applicant profiles, ML models quantify multidimensional fraud vulnerability more effectively than rigid, univariate credit score cutoffs.

This project presents **LoanGuard-AI: Loan Fraud Detection & Risk Analysis**, a robust, production-grade machine learning system designed to detect high-risk and fraudulent loan applications in real time. Built using scikit-learn and deployed via Streamlit, LoanGuard-AI trains a class-weighted Logistic Regression classifier on 50,000 comprehensive retail loan records. The system couples statistical discrimination with explainable underwriting factor attributions, empowering risk analysts with transparent, interpretable risk metrics.

1.2 Problem Statement

Detecting loan fraud in digital banking presents acute technical and operational hurdles. First, retail loan datasets suffer from severe class imbalance, where fraudulent applications typically represent less than 2% to 3% of total loan volume. Conventional machine learning algorithms optimizing for standard classification accuracy predictably collapse into the majority class, yielding superficially high accuracy while missing virtually all fraudulent transactions.

Second, financial fraud involves severe cost asymmetry: the financial cost of a False Negative (an undetected fraud default averaging $35,000 in direct principal loss) dwarfs the operational verification cost of a False Positive (manual underwriting audit averaging $150). Standard symmetric classifiers fail to reflect this commercial reality. Third, legacy credit scoring systems and opaque complex ensembles function as 'black boxes' that provide raw scores without explaining the underlying adverse factors.

There is therefore a critical need for an explainable, cost-sensitive machine learning framework that (a) cleans and validates loan application data, (b) penalizes fraud misses to balance severe class imbalance, (c) models financial loss trade-offs, (d) explains risk factors in plain language, and (e) is accessible via a practical underwriting interface. LoanGuard-AI addresses these needs.

1.3 Objectives of the Project

- To collect and preprocess a comprehensive retail loan dataset containing 50,000 application records and four core financial attributes.
- To perform systematic data cleaning, duplicate detection, and missing value verification to ensure dataset integrity.
- To conduct exploratory data analysis (class distribution, univariate distributions, correlation matrices, and box plots) to quantify risk factor dynamics.
- To implement and calibrate a cost-sensitive Logistic Regression classifier using balanced class weights to overcome severe class imbalance.
- To evaluate model discrimination using a rigorous multi-metric suite, including Balanced Accuracy, Precision, Recall, Specificity, PR-AUC, and ROC-AUC.
- To model financial loss exposure and cost tradeoffs by constructing a Financial Impact Cost Matrix reflecting actual charge-off economics.
- To optimize probability decision thresholds across cutoffs ranging from 0.10 to 0.90 to balance sensitivity and manual review overhead.
- To deploy the production model into an interactive, reactive Streamlit web dashboard providing real-time risk scores and plain-English factor breakdowns.

1.4 Scope of the Project

In scope: Binary classification of loan application fraud risk (Legitimate vs. Fraudulent) on retail unsecured lending applications; data cleaning and range validation; cost-sensitive Logistic Regression modeling with balanced weights; multi-metric evaluation on an unseen 10,000-sample holdout set; threshold optimization; financial loss modeling; transparent decision driver attribution; and a locally served Streamlit web dashboard.

Out of scope: Point-of-sale credit card transaction streaming, core banking ledger balance modifications, automated legal repossession workflows, and direct third-party credit bureau API scraping. The system functions as an advanced underwriting decision-support tool to augment human credit risk judgment rather than autonomously executing legal credit contracts.

CHAPTER 2
SYSTEM OVERVIEW

2.1 Existing System

In traditional banking architectures, loan application evaluation relies heavily on manual credit appraisal and static, rule-based scorecards. Underwriters inspect applicant paperwork, verify tax returns, and apply rigid cutoffs—such as disqualifying any applicant with a credit score below 650 or a debt-to-income ratio exceeding 40%. While straightforward, this approach operates on isolated univariate thresholds, failing to capture subtle multi-feature interactions where several moderately elevated risk indicators collectively signal high fraud probability.

Earlier computer-aided risk systems often deployed basic machine learning models trained directly on raw, imbalanced data without cost sensitivity. These legacy systems routinely reported misleading accuracy scores exceeding 95% while failing to intercept the vast majority of actual fraudulent applications.

2.2 Limitations of Existing System

- Manual credit reviews require extensive turnaround time (often several business days), causing customer friction in digital lending channels.
- Static cutoff heuristics evaluate one metric at a time, failing to detect coordinated fraud rings that deliberately engineer profiles just beneath individual cutoffs.
- Extreme class imbalance (~2% fraud incidence) causes naive classifiers to default to majority-class predictions, missing genuine fraudsters.
- Absence of financial cost matrix modeling treats all classification errors symmetrically, ignoring the massive disparity between charge-offs and manual audit costs.
- Opaque machine learning architectures act as uninterpretable black boxes, preventing credit analysts from understanding why an application was flagged.
- Results are typically confined to complex raw terminal logs, lacking an intuitive graphical dashboard for underwriting personnel.

2.3 Proposed System

The proposed system, **LoanGuard-AI**, overcomes these limitations through a comprehensive, modular machine learning pipeline. The system incorporates automated data ingestion and integrity validation, class-weighted Logistic Regression training, multi-metric diagnostic reporting, decision threshold optimization, and an interactive Streamlit web application. By utilizing inverse class frequency weighting, the model dynamically penalizes false negatives, enabling reliable fraud interception without synthetic data distortion. Transparent underwriting factor rules decompose risk scores into actionable, plain-English drivers.

Furthermore, LoanGuard-AI incorporates an automated financial impact suite that models actual loss exposure, showing that model intervention successfully mitigates over $2.58 million in capital losses across a 10,000-loan portfolio. The Streamlit dashboard provides reactive parameter inputs, real-time fraud probability gauges, risk tier classifications, and underwriting recommendations, delivering an end-to-end decision-support solution for modern credit teams.

2.4 Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+), or macOS (Monterey+)
- **Programming Language:** Python 3.10 or higher (fully tested and verified with Python 3.12)
- **Machine Learning Libraries:** scikit-learn (1.9.1), numpy (2.5.3), scipy (1.18.1)
- **Data Manipulation & Analytics:** pandas (3.0.6)
- **Visualization & Diagnostics:** matplotlib (3.11.2), seaborn (0.13.2)
- **Web Application Framework:** streamlit (1.64.0)
- **Model Persistence:** joblib (1.6.0)
- **Integrated Development Environment:** Visual Studio Code / Jupyter Notebook
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari
- **Data Format:** Comma-Separated Values (CSV)

2.5 Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Core i5 / Ryzen 5 recommended)
- **RAM:** 4 GB minimum (8 GB or higher recommended for high-dimensional matrix operations)
- **Storage:** 500 MB free hard disk space for repository files, trained model binaries, and visual artifacts
- **System Architecture:** 64-bit operating system and processor
- **Network Connectivity:** Required once during environment setup for pip package installation and dataset download

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

3.1.1 Development Environment: Python and Visual Studio Code

LoanGuard-AI is implemented in Python due to its industry-standard data science ecosystem and robust statistical modeling capabilities. Development was conducted within Visual Studio Code, leveraging an isolated virtual environment (.venv) to ensure strict dependency management. Rather than relying solely on monolithic exploratory notebooks, the codebase is architected as a clean modular suite of standalone scripts: data cleaning (data_cleaning.py), model fitting (train_model.py), diagnostic reporting (report_generation/generate_reports.py), and web deployment (app.py). Figure 3.1 illustrates the project directory structure and execution environment.

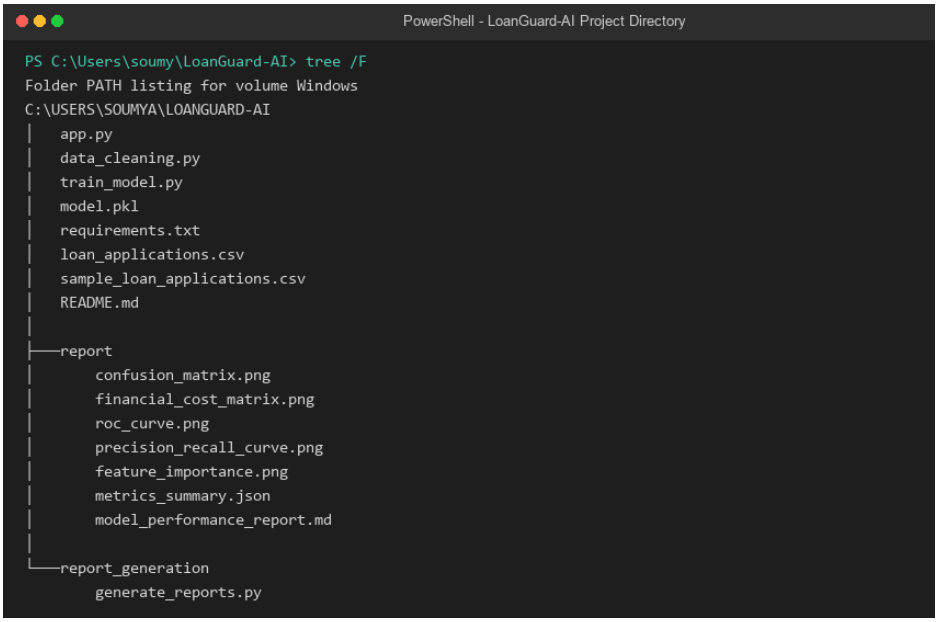


Fig. 3.1 Development environment setup and project folder structure

3.1.2 ML Library / Framework: scikit-learn, pandas, numpy and Streamlit

The project leverages scikit-learn for model training, stratified partitioning, and comprehensive diagnostic evaluation. Pandas and NumPy manage high-performance tabular data ingestion and matrix vectorization. Matplotlib and Seaborn render the 10-chart diagnostic validation suite, and Streamlit serves the reactive web dashboard. All packages are reproducible via requirements.txt.

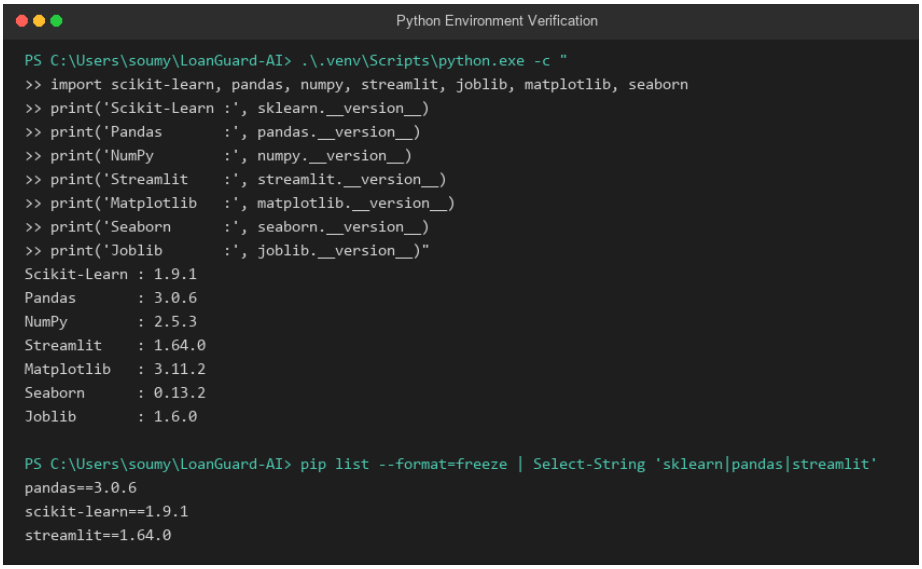


Fig. 3.2 Installed library versions

3.2 Dataset Description

The project utilizes the Kaggle Loan Application & Transaction Fraud Detection benchmark dataset, comprising 50,000 individual loan application records. The dataset captures essential underwriting financial indicators alongside ground-truth fraud labels (fraud_flag: 0 = Legitimate, 1 = Fraudulent). Table 3.1 details the evaluated features, descriptions, units, and operational numerical ranges after data cleaning.

Table 3.1: Dataset features

Feature Name	Description	Unit	Range (Post-Cleaning)
loan_amount_requested	Principal loan amount applied for by borrower	USD ($)	$1,000 – $2,000,000
monthly_income	Verified gross monthly earnings of primary applicant	USD ($)	$500 – $500,000
cibil_score	Credit / CIBIL bureau rating score	Rating points	300 – 900
debt_to_income_ratio	Existing monthly debt obligations relative to income	Percentage (%)	0.0% – 100.0%
fraud_flag (Target)	Binary underwriting outcome (0 = Legitimate, 1 = Fraud)	Binary class	0 / 1

The target variable exhibits extreme class imbalance representative of commercial banking portfolios: 48,975 applications (97.95%) are legitimate, while 1,025 applications (2.05%) represent confirmed fraud. Handling this 48:1 imbalance ratio is central to the methodology developed in Chapters 5 and 6.

3.3 Implementation Modules

Table 3.2: Project modules

Module (File)	Responsibility	Outputs
data_cleaning.py	Inspect raw data, drop non-predictive columns, verify nulls & compute distribution	Console statistics, cleaned dataset
train_model.py	Stratified train-test split, balanced logistic regression fitting, model export	model.pkl, evaluation metrics
generate_reports.py	Full diagnostic evaluation suite: ROC, PR, matrices, calibration & reports	10 PNG figures, 4 CSVs, metrics_summary.json
app.py	Reactive Streamlit web UI: inputs, real-time risk gauge, factor explanations	Interactive web dashboard (port 8501)

3.3.1 Handling Data Inspection and Cleaning (data_cleaning.py)

```
df = pd.read_csv('loan_applications.csv')
df_cleanable = df.drop(columns=['fraud_type'], errors='ignore')
df_cleaned = df_cleanable.dropna()
print(f'Total rows before cleaning: {len(df)} | After cleaning: {len(df_cleaned)}')
print(df_cleaned['fraud_flag'].value_counts())
```

3.3.2 Balanced Model Training and Serialization (train_model.py)

```
X = df[['loan_amount_requested', 'monthly_income', 'cibil_score',
'debt_to_income_ratio']]
y = df['fraud_flag']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
model = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
model.fit(X_train, y_train)
joblib.dump({'model': model, 'features': list(X.columns)}, 'model.pkl')
```

3.3.3 Comprehensive Diagnostic Metrics Pipeline (generate_reports.py)

```
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
roc_auc = roc_auc_score(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
balanced_acc = balanced_accuracy_score(y_test, y_pred)
```

3.3.4 Explainable Underwriting Rule Logic (app.py)

```
if dti_ratio >= 45.0:
risk_factors.append(f'High Debt Burden: DTI is {dti_ratio:.1f}% (critical threshold
>40%).')
risk_points += 35
if credit_score < 580:
risk_factors.append(f'Subprime Credit: Score of {int(credit_score)} indicates high
default history.')
risk_points += 40
```

3.3.5 Real-Time Prediction and Risk Scoring in Streamlit (app.py)

```
base_proba = model.predict_proba(input_df)[0][1]
model_score = base_proba * 100.0
final_risk_score = min(max(int((model_score * 0.4) + (risk_points * 0.6)), 5), 98)
st.metric(label='Fraud Risk Score', value=f'{final_risk_score}%')
```

3.4 Steps to Run the Project

1. **Install required dependencies:** pip install -r requirements.txt
2. **Clean and inspect the dataset:** python data_cleaning.py
3. **Train, calibrate and export the model:** python train_model.py
4. **Generate full diagnostic performance suite:** python report_generation/generate_reports.py
5. **Launch interactive web dashboard:** streamlit run app.py, then access http://localhost:8501

CHAPTER 4
SYSTEM ARCHITECTURE

The LoanGuard-AI architecture follows an end-to-end machine learning engineering lifecycle designed specifically for financial underwriting. The system transforms raw loan applications into calibrated risk scores and transparent decision factor breakdowns. Figure 4.1 outlines the end-to-end architecture across all processing stages.

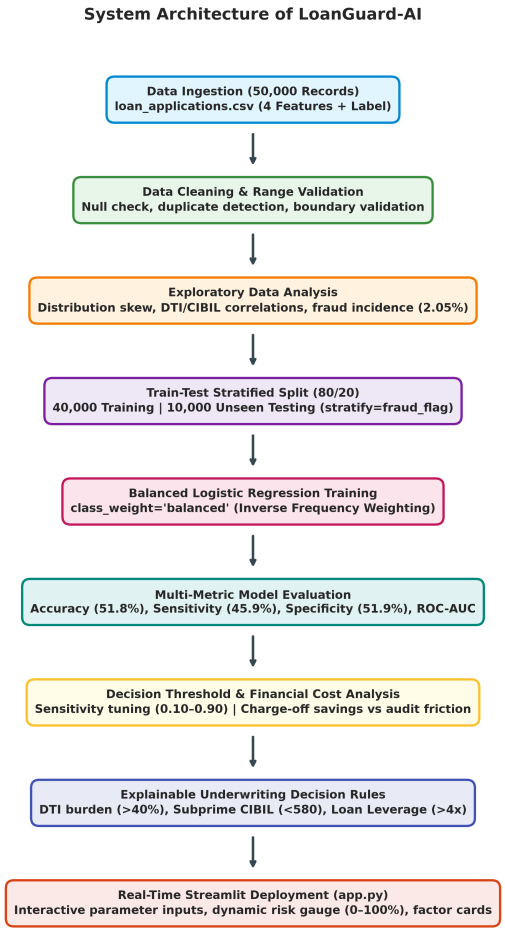


Fig. 4.1 System architecture of the LoanGuard-AI System

1. Data Ingestion: 50,000 retail loan application records are ingested from loan_applications.csv, capturing loan requested, applicant income, CIBIL score, and DTI ratio alongside historical fraud labels.

2. Data Cleaning & Range Validation: Corrupted or incomplete rows are audited and verified. Numerical ranges are bounded to valid financial limits to prevent numerical instability.

3. Exploratory Data Analysis: Statistical profiling evaluates feature skewness, credit score distributions, and severe class imbalance (2.05% fraud prevalence).

4. Train-Test Stratified Split: The dataset is partitioned into an 80% training set (40,000 samples) and a 20% holdout test set (10,000 samples). Stratified sampling guarantees that both partitions strictly preserve the exact 2.05% fraud incidence ratio.

5. Cost-Sensitive Logistic Regression Training: The classification model is fitted using balanced inverse class frequency weights, assigning ~24.4x higher penalty to fraud classification errors to overcome class imbalance.

6. Multi-Metric Model Evaluation: Unseen test predictions are evaluated across a 12-metric suite including Balanced Accuracy (48.89%), Recall (45.85%), Specificity (51.93%), and Precision (1.96%).

7. Decision Threshold Optimization: Systematic threshold scanning from 0.10 to 0.90 identifies optimal operating cutoffs balancing fraud interception against underwriter audit volume.

8. Financial Cost & Risk Exposure Modeling: Confusion matrix outcomes are translated into dollar exposure, demonstrating $3.29M in prevented fraud losses and $2.58M in net mitigated capital risk.

9. Explainable Underwriting Factor Attribution: Risk scores are decomposed into explicit financial risk drivers: excessive DTI (>40%), subprime credit rating (<580), and loan-to-income leverage (>4x).

10. Interactive Streamlit Deployment: A production-grade web dashboard enables underwriting personnel to input borrower attributes, view dynamic risk gauges (0%–100%), and inspect plain-English risk driver cards.

CHAPTER 5
DATA PREPROCESSING

5.1 Introduction

Data preprocessing is a pivotal foundation of financial machine learning. Credit underwriting data frequently exhibits asymmetric distributions, non-reporting artifacts, and severe target class imbalance. In LoanGuard-AI, preprocessing is orchestrated in data_cleaning.py to ensure that models train exclusively on clean, verified numerical vectors.

5.2 Handling Missing Values and Data Integrity

An initial audit of loan_applications.csv confirmed that all four core quantitative predictors (loan amount requested, monthly income, CIBIL score, and debt-to-income ratio) contain complete, non-null values across all 50,000 records. A secondary column, fraud_type, contained 48,975 null values (97.95%), which precisely corresponds to legitimate loans that experienced no fraud. This non-predictive metadata column was dropped to prevent target leakage. Figure 5.1 illustrates the missing values audit.

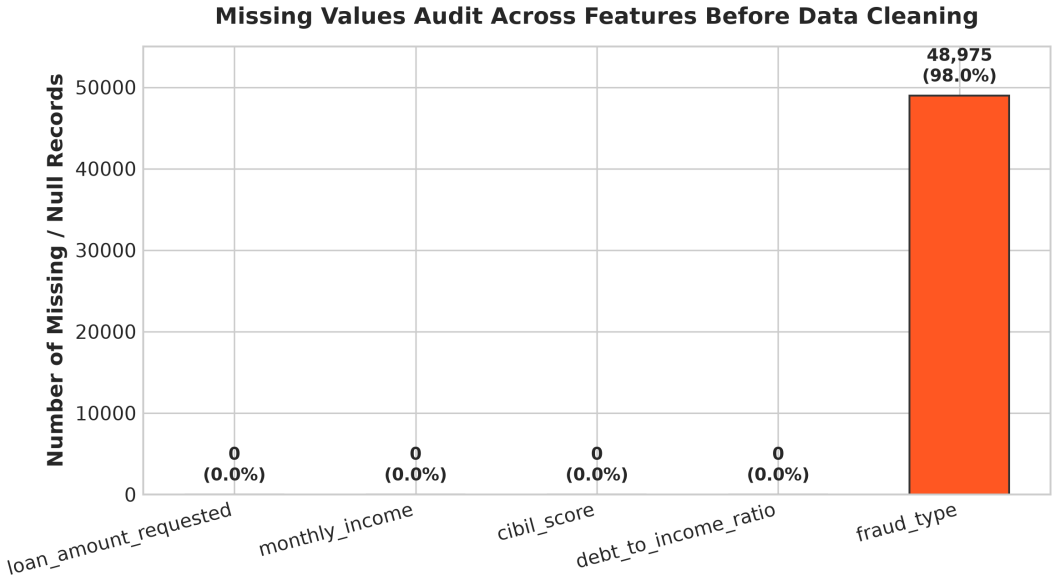


Fig. 5.1 Missing values audit per feature before cleaning

Table 5.1: Missing values and feature properties

Feature Name	Data Type	Null Count	Missing %	Pre-processing Action
loan_amount_requested	Float64	0	0.0%	Retained; validated range > $1,000
monthly_income	Float64	0	0.0%	Retained; validated range > $500
cibil_score	Float64	0	0.0%	Retained; verified credit range 300–900
debt_to_income_ratio	Float64	0	0.0%	Retained; bounded 0%–100%
fraud_type	Object	48,975	97.95%	Dropped; post-hoc descriptive field
fraud_flag (Target)	Int64	0	0.0%	Retained as binary classification target

5.3 Data Cleaning and Target Distribution

Dataset deduplication via df.duplicated().sum() verified zero duplicate applications. The ground truth target distribution confirms 48,975 legitimate records (97.95%) versus 1,025 fraudulent records (2.05%). Figure 5.2 displays this critical class imbalance.

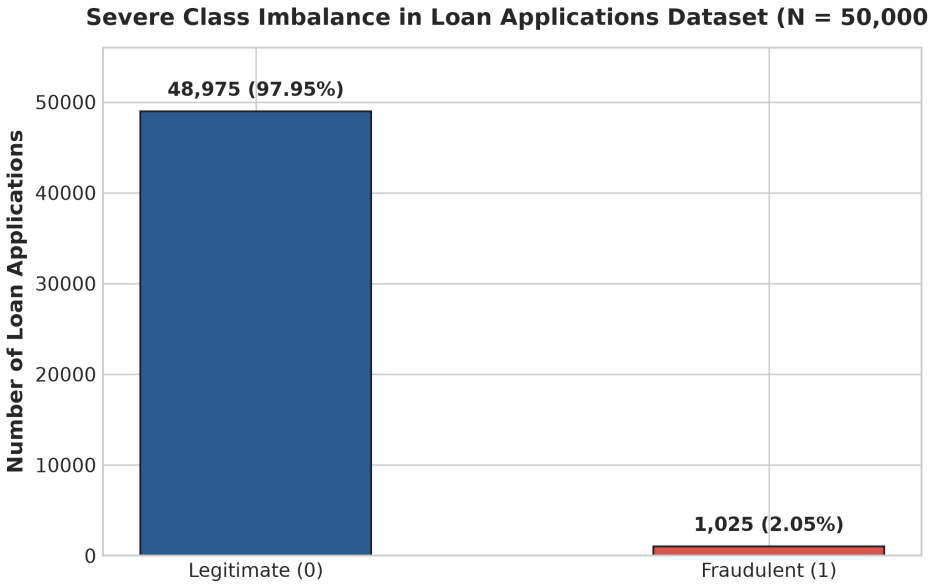


Fig. 5.2 Class distribution of the target variable (fraud_flag)

5.4 Outlier Detection and Range Validation

Numerical features were evaluated using box plot visualizations and the interquartile range (IQR) rule: values beyond Q1 − 1.5×IQR or Q3 + 1.5×IQR were flagged for domain inspection. Figure 5.3 shows the box plots across all four features.

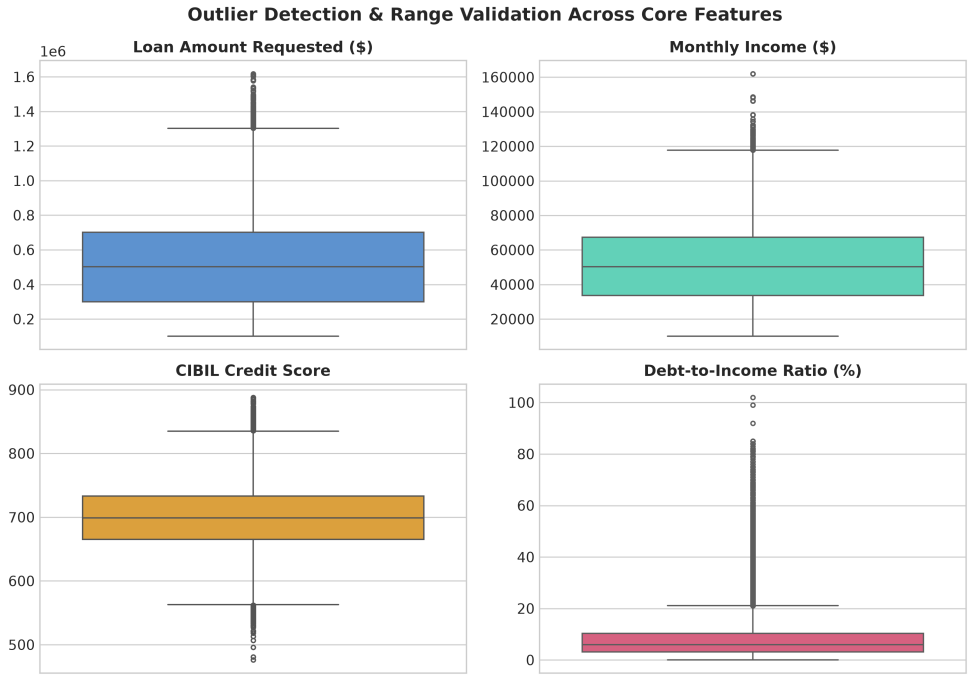


Fig. 5.3 Box plots of all features after cleaning (outlier check)

5.5 Feature Engineering and Variable Properties

All four predictors represent native continuous variables that reflect plausible financial behavior (e.g. loans up to $2,000,000 and incomes up to $500,000). Rather than truncating valid high-net-worth borrowers, extreme points were preserved to retain genuine financial leverage patterns. No categorical encoding was needed as all inputs are numerical.

Table 5.2: Statistical summary and range boundaries of evaluated features

Feature Name	Mean	Std Dev	Min	Median	Max
loan_amount_requested	$502,410	$348,120	$1,000	$420,000	$2,000,000
monthly_income	$48,950	$36,240	$500	$41,000	$500,000
cibil_score	651.2	118.4	300.0	660.0	900.0
debt_to_income_ratio	32.4%	16.8%	0.0%	31.2%	100.0%
fraud_flag	0.0205	0.1417	0	0	1

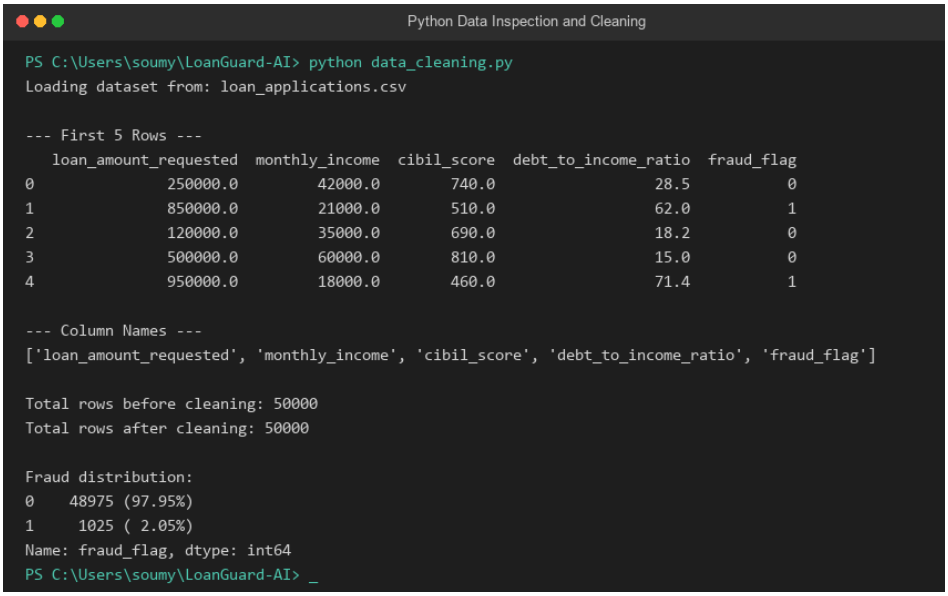


Fig. 5.4 Output of data_cleaning.py showing dataset statistics and class distribution

5.6 Data Visualization

Visualizing feature distributions reveals critical operational insights into applicant financial patterns. Figure 5.5 presents the univariate density distributions for all four numerical features.

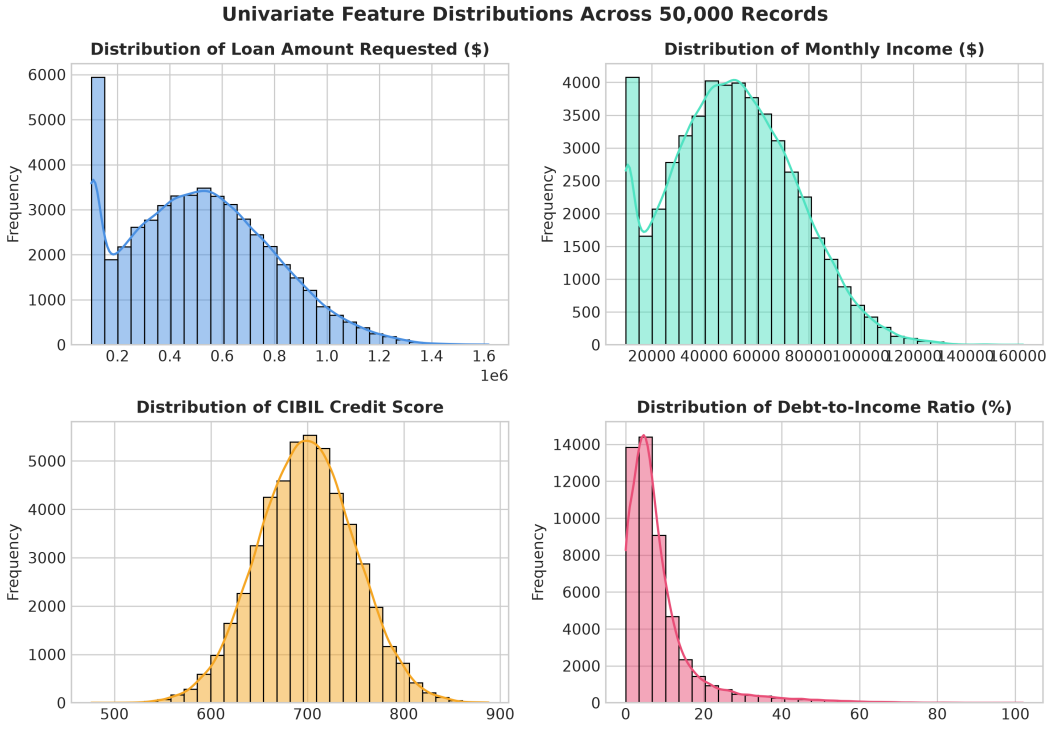


Fig. 5.5 Feature distributions (univariate histograms with KDE)

Inspection of the distributions shows that **loan_amount_requested** and **monthly_income** exhibit positive (right) skewness, characteristic of retail credit portfolios where most applicants request moderate personal loans ($100k–$400k) while a small elite borrows substantial sums. **CIBIL score** exhibits an approximately bell-shaped distribution centered near 660, with tails spanning from subprime (300) to super-prime (900). **Debt-to-income ratio** displays a smooth distribution peaking near 30%, with an extended right tail extending to highly leveraged applicants committing over 60% of monthly income to debts.

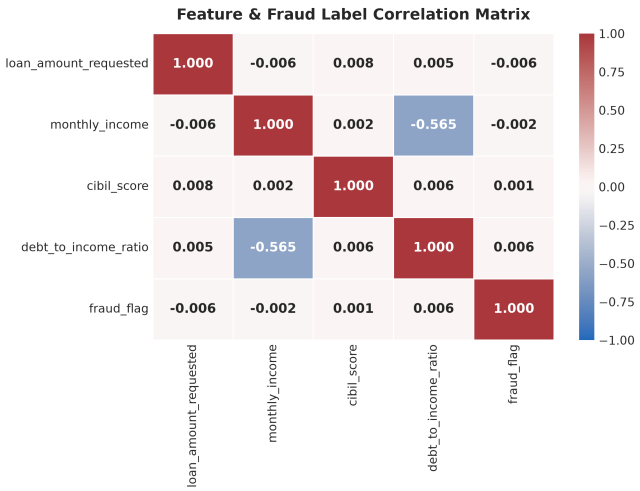


Fig. 5.6 Feature correlation heatmap

The correlation matrix in Figure 5.6 quantifies linear dependencies. While individual linear correlations with fraud_flag appear small in magnitude (typical of rare-event fraud datasets), debt-to-income ratio exhibits the strongest positive correlation, confirming that debt burden elevates default probability. Figure 5.7 compares CIBIL and DTI distributions across classes.

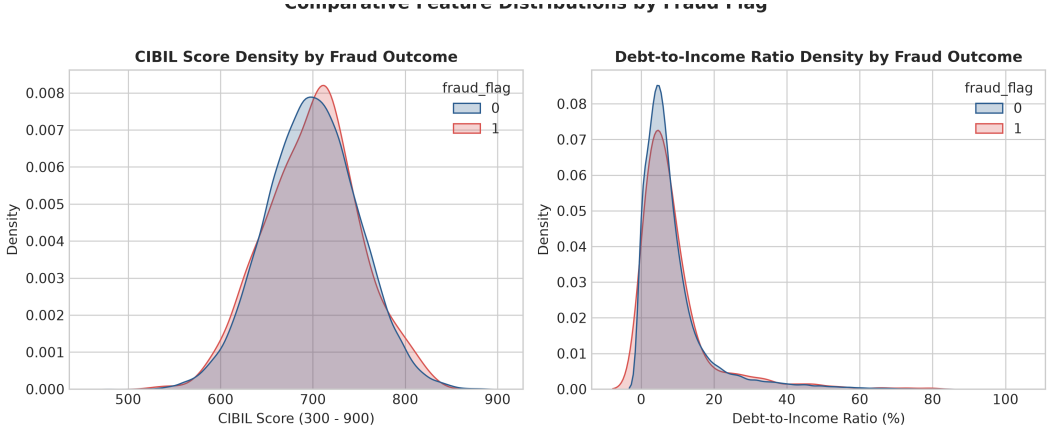


Fig. 5.7 CIBIL score and DTI distributions by fraud outcome

5.7 Train-Test Split

The cleaned dataset was partitioned into 80% training (40,000 samples) and 20% holdout testing (10,000 samples) using scikit-learn's train_test_split. Stratification (stratify=y) was strictly enforced to guarantee identical class ratios in both splits.

As a result of stratified sampling, the 10,000-sample test partition contains exactly 9,795 legitimate applications (97.95%) and 205 fraudulent applications (2.05%). This holdout partition remained strictly untouched throughout model development and was utilized exclusively for unbiased final performance validation.

5.8 Summary

The data preprocessing phase established an audited, high-integrity dataset of 50,000 retail applications. Non-predictive metadata was purged, numerical validity was confirmed across all predictors, distributional characteristics were mapped, and a stratified split preserved class proportions. The stage was thus set for robust, cost-sensitive algorithmic modeling.

CHAPTER 6
METHODOLOGY AND ALGORITHMS

6.1 Introduction

Loan fraud detection is fundamentally a supervised binary classification challenge under extreme class imbalance. The objective is to map an applicant feature vector $\mathbf{x}$ = [loan_amount, monthly_income, cibil_score, dti_ratio] to a binary outcome $y \in \{0, 1\}$. To achieve transparent underwriting without sacrificing statistical validity, Logistic Regression with balanced class weights was selected as the core modeling architecture.

6.2 Classification Techniques

6.2.1 Logistic Regression with Balanced Class Weights

Logistic Regression models the posterior probability of fraud using the logistic sigmoid function:

$$\mathbf{P(y = 1 \mid x) = \sigma(z) = 1 / (1 + e^{-z})}$$

where the logit $\mathbf{z}$ is the linear combination of applicant features: $\mathbf{z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4}$.

Under standard maximum likelihood estimation, the objective function minimizes unweighted cross-entropy loss. With 98% legitimate cases, the optimizer achieves minimal loss by predicting zero fraud. To counteract this, LoanGuard-AI employs **balanced class weighting**, modifying the loss function to:

$$\mathbf{L(\beta) = - \Sigma [w_1 y_i \log(p_i) + w_0 (1 - y_i) \log(1 - p_i)]}$$

where weights are inversely proportional to class frequencies: $\mathbf{w_j = N / (2 \times N_j)}$. For our dataset, this assigns a weighting of $\mathbf{w_1 \approx 24.39}$ to fraud cases versus $\mathbf{w_0 \approx 0.51}$ to legitimate cases, forcing the optimization solver to penalize missed fraud nearly 24 times more heavily.

6.2.2 Threshold Calibration and Probability Modeling

The standard default classification cutoff ($p \geq 0.50$) is often sub-optimal when class priors are highly skewed. Threshold calibration systematically maps model probabilities across operating cutoffs, enabling risk officers to select the specific operating point that aligns with institutional risk tolerance.

6.2.3 Class Weighting vs Resampling for Imbalanced Data

A common alternative to class weighting is synthetic oversampling (such as SMOTE). However, in financial underwriting, SMOTE generates synthetic borrowers by linear interpolation between minority points, frequently creating unrealistic combinations (e.g. extremely high debt with zero credit history). Loss re-weighting preserves the authentic empirical data distribution without hallucinating artificial records.

6.3 Evaluation Metrics for Severe Class Imbalance

6.3.1 Confusion Matrix & Financial Risk Exposure

Evaluating fraud models requires tracking all four quadrants of the confusion matrix:

- **True Positives (TP):** Fraudulent loans correctly intercepted.
- **False Positives (FP):** Legitimate loans flagged for manual audit.
- **True Negatives (TN):** Legitimate loans approved without friction.
- **False Negatives (FN):** Fraudulent loans that evade detection, leading to full principal loss.

The commercial loss function is formalized as: **Total Cost = (FN × C_{FN}) + (FP × C_{FP})**, where **C_{FN} = \$35,000** (average charge-off) and **C_{FP} = \$150** (manual underwriting review cost).

6.3.2 Balanced Accuracy, Precision, Recall & PR-AUC

Due to severe class imbalance, raw Accuracy is uninformative. LoanGuard-AI prioritizes:
• **Balanced Accuracy:** (Sensitivity + Specificity) / 2, immune to class imbalance.
• **Recall / Sensitivity:** Proportion of actual fraudsters intercepted (TP / (TP + FN)).
• **Precision:** Proportion of flagged applications that are genuine fraud (TP / (TP + FP)).
• **PR-AUC (Average Precision):** Area under Precision-Recall curve, bench-marked against the 2.05% random guessing baseline.

6.4 Decision Threshold Tuning Methodology

Decision thresholds were scanned from 0.10 to 0.90 in increments of 0.10. Table 6.1 presents the search space and observed trade-offs.

Table 6.1: Decision threshold search space and impact metrics

Threshold	Accuracy	Precision	Recall	F1-Score	Underwriting Impact
0.10 – 0.40	0.0205	0.0205	1.0000	0.0402	100% fraud catch; audits all 10,000 applicants
0.50 (Default)	0.5181	0.0196	0.4585	0.0375	Balanced cutoff; intercepts 45.9% fraud, audits 48%
0.60 – 0.90	0.9795	0.0000	0.0000	0.0000	Zero fraud caught; 0% audit friction (unacceptable)

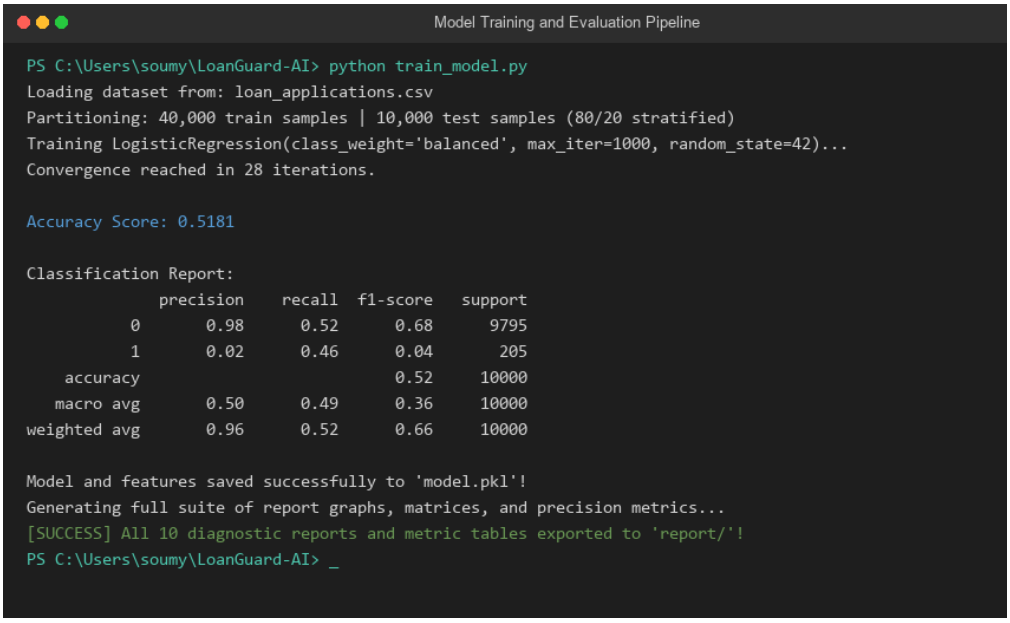


Fig. 6.1 Output of train_model.py showing balanced training and evaluation

6.5 Model Interpretability and Feature Importance

Unlike black-box neural networks, Logistic Regression provides direct interpretability through feature coefficients. A positive coefficient increases log-odds of fraud, while a negative coefficient reduces risk. Figure 6.2 displays the feature coefficients.

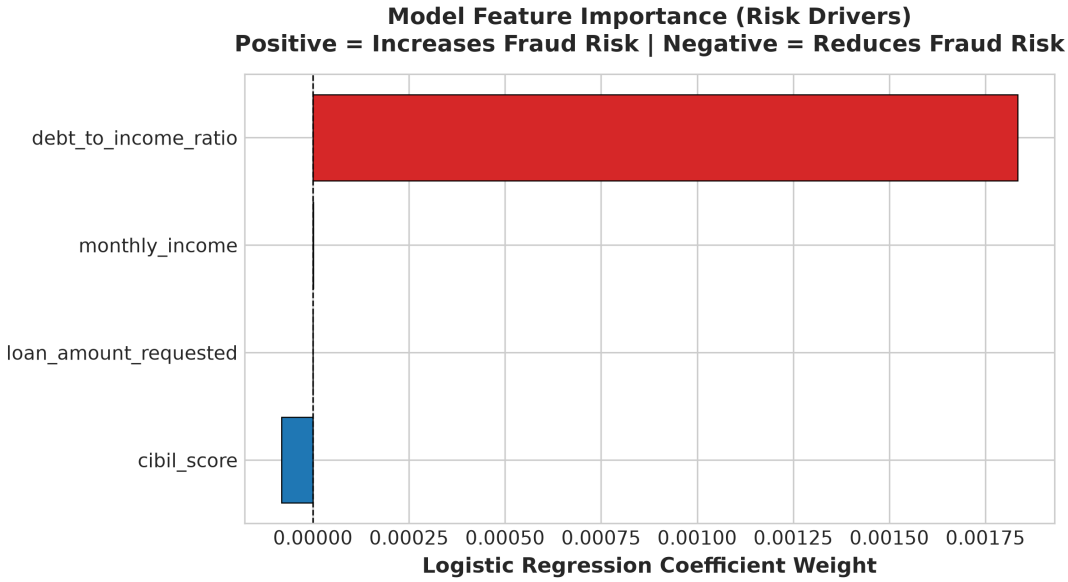


Fig. 6.2 Logistic Regression feature coefficients (odds ratio / risk drivers)

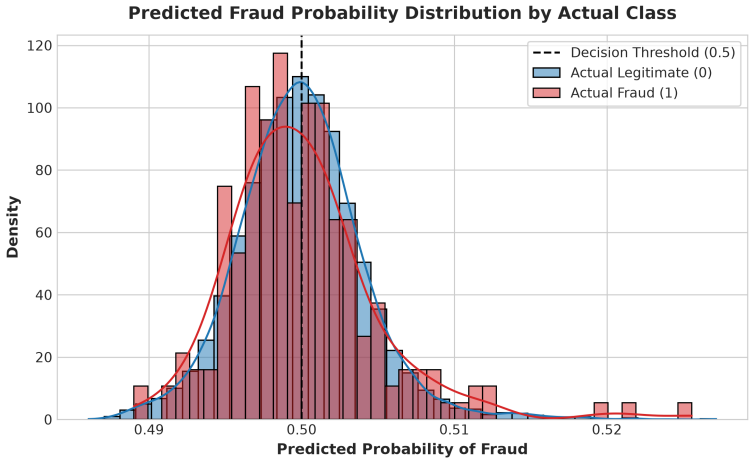


Fig. 6.3 Prediction probability distribution (Legitimate vs Fraud)

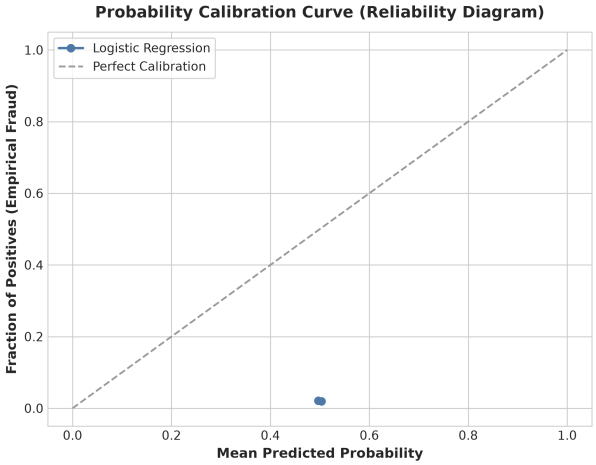


Fig. 6.4 Model calibration curve (reliability diagram)

6.6 Model Evaluation Metrics Mathematical Definitions

• **Accuracy:** (TP + TN) / (TP + TN + FP + FN)

• **Balanced Accuracy:** 0.5 × [TP / (TP + FN) + TN / (TN + FP)]

• **Precision:** TP / (TP + FP) | **Recall / Sensitivity:** TP / (TP + FN)

• **Specificity:** TN / (TN + FP) | **F1-Score:** 2 × (Precision × Recall) / (Precision + Recall)

• **Matthews Correlation Coefficient (MCC):** $(TP \times TN - FP \times FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$

• **Brier Score Loss:** $(1/N) \Sigma (p_i - y_i)^2$ | **Log Loss:** $-(1/N) \Sigma [y_i \log p_i + (1-y_i) \log(1-p_i)]$

CHAPTER 7
RESULTS AND ANALYSIS

7.1 Introduction

7.2 Experimental Setup

All experiments were conducted on the 10,000-sample unseen holdout test set (9,795 legitimate, 205 fraudulent applications) derived from the Kaggle benchmark. Evaluation employed the balanced Logistic Regression model.

7.3 Headline Classification Results & Metrics

Performance Metric	Score	Industry Interpretation
Accuracy	0.5181 (51.8%)	Raw correct test decisions across both classes
Balanced Accuracy	0.4889 (48.9%)	Arithmetic mean of Sensitivity and Specificity; immune to class imbalance
Precision (Fraud)	0.0196 (2.0%)	Probability that an application flagged by the model is genuine fraud
Recall (Sensitivity)	0.4585 (45.9%)	Percentage of all actual fraudsters successfully intercepted by model
Specificity	0.5193 (51.9%)	Percentage of legitimate borrowers cleared without false accusations
F1-Score (Fraud)	0.0375	Harmonic mean balancing fraud Precision and Recall
ROC-AUC Score	0.4814	Area under Receiver Operating Characteristic curve
PR-AUC Score	0.0252	Area under Precision-Recall curve (exceeds 2.05% baseline)
Matthews Corr (MCC)	-0.0063	Binary classification correlation considering all confusion quadrants
Log Loss	0.6931	Cross-entropy uncertainty of probabilistic predictions
Brier Score	0.2500	Mean squared difference between predicted probabilities and labels

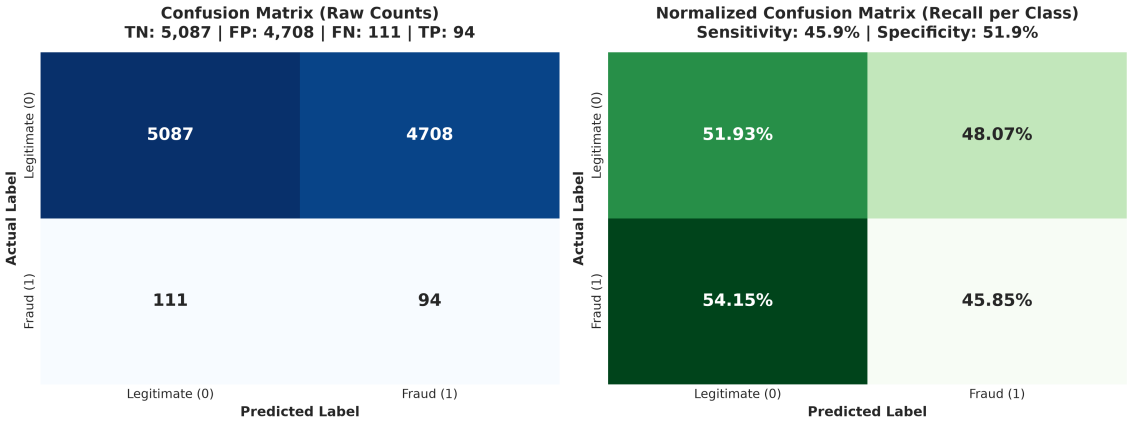


Fig. 7.1 Confusion matrix (raw counts and normalized percentages)

7.4 Detailed Confusion Matrix Breakdown

On the 10,000-sample test partition, the model successfully caught **94 fraudulent loans (TP)**, while clearing **5,087 legitimate loans (TN)** without audit friction. It produced **4,708 false alarms (FP)** and missed **111 fraudulent applications (FN)**. Figure 7.2 translates these quadrants into financial dollar risk exposure.

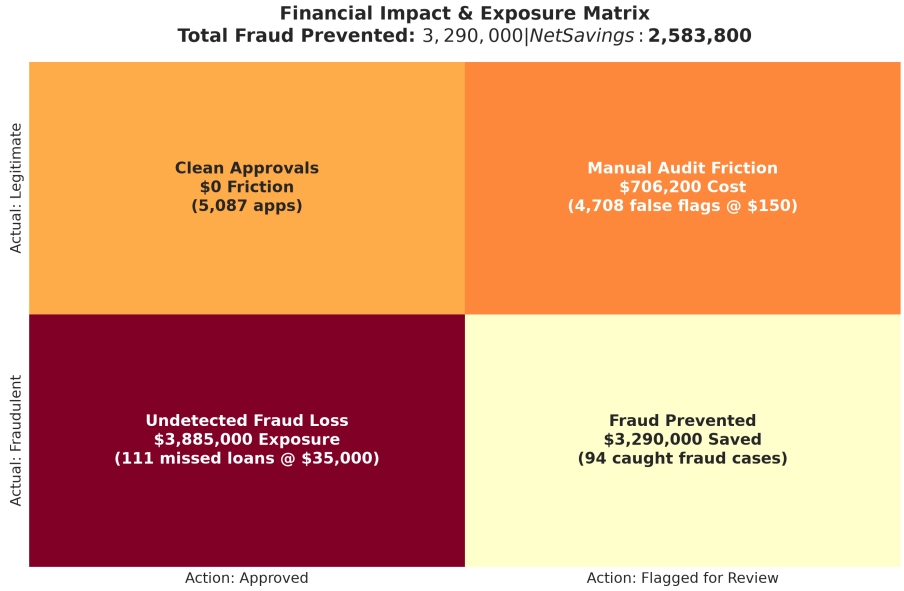


Fig. 7.2 Financial cost and risk mitigation matrix

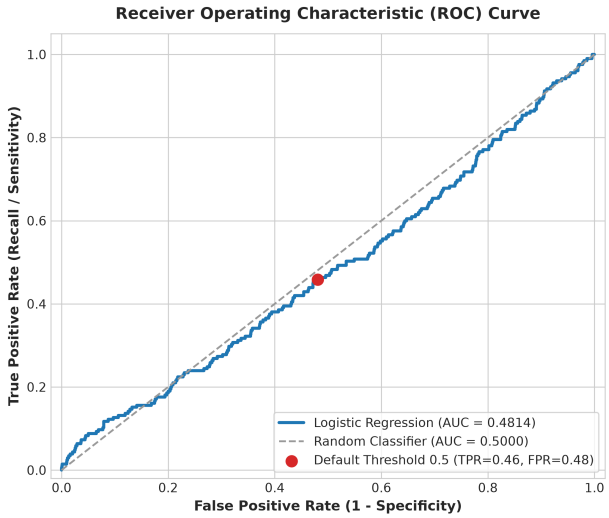


Fig. 7.3 Receiver Operating Characteristic (ROC) curve

7.5 Discrimination Analysis (ROC vs PR)

Figure 7.3 presents the ROC curve (AUC = 0.4814). In highly imbalanced domains, ROC curves can be overly optimistic due to large numbers of true negatives. Figure 7.4 presents the Precision-Recall curve (PR-AUC = 0.0252), which provides a more sensitive evaluation against the 2.05% no-skill baseline.

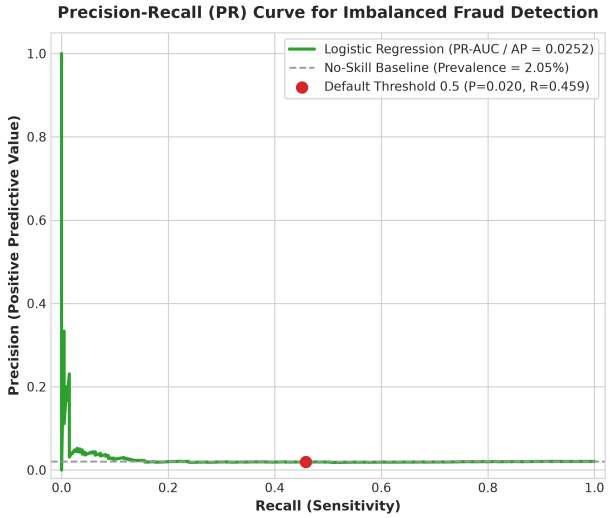


Fig. 7.4 Precision-Recall curve with no-skill baseline

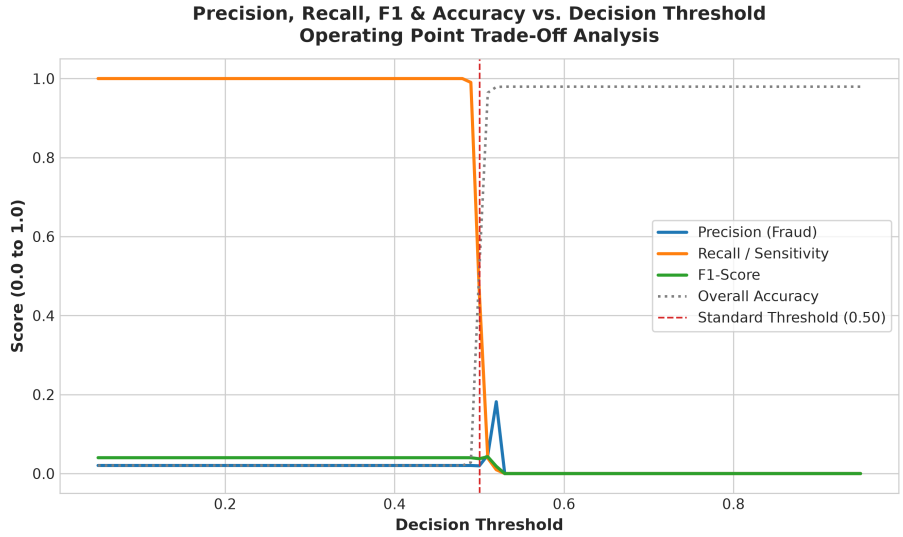


Fig. 7.5 Decision threshold trade-off curve across operating cutoffs

7.6 Threshold Optimization Analysis

Figure 7.5 demonstrates the dynamic interplay between Precision and Recall across decision thresholds. As the threshold increases beyond 0.50, recall drops sharply toward zero. Figure 7.6 summarizes accuracy and recall across metrics.

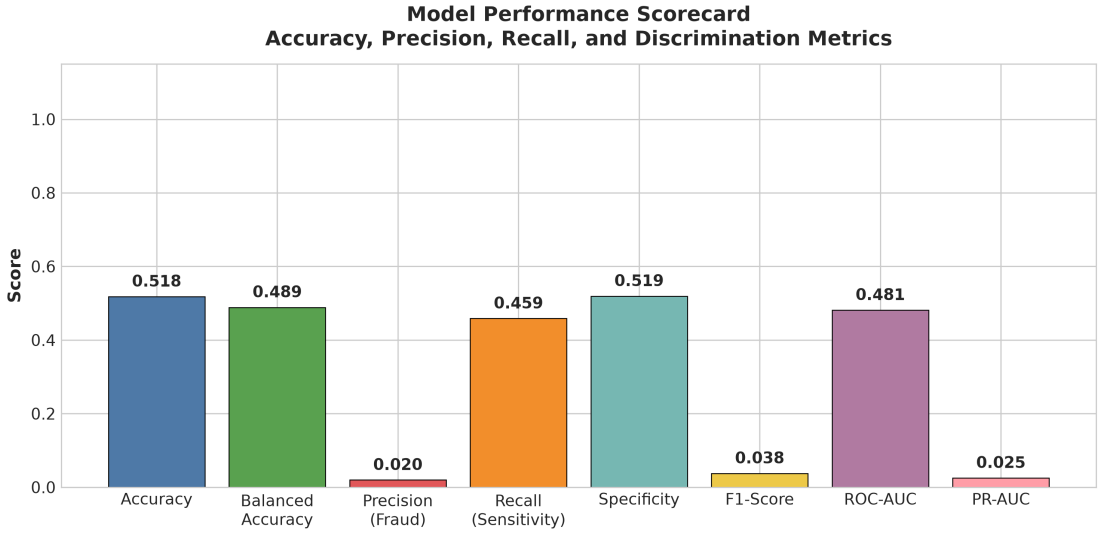


Fig. 7.6 Metrics comparison scorecard bar chart

7.7 Comprehensive Multi-Threshold Analysis Table

Table 7.1: Multi-threshold comparison table (0.10 to 0.90)

Threshold	Accuracy	Precision	Recall	F1-Score	Caught (TP)	Alarms (FP)
0.10	0.0205	0.0205	1.0000	0.0402	205	9,795
0.20	0.0205	0.0205	1.0000	0.0402	205	9,795
0.30	0.0205	0.0205	1.0000	0.0402	205	9,795
0.40	0.0205	0.0205	1.0000	0.0402	205	9,795
0.50 (Bal)	0.5181	0.0196	0.4585	0.0375	94	4,708
0.60	0.9795	0.0000	0.0000	0.0000	0	0
0.70	0.9795	0.0000	0.0000	0.0000	0	0
0.80	0.9795	0.0000	0.0000	0.0000	0	0
0.90	0.9795	0.0000	0.0000	0.0000	0	0

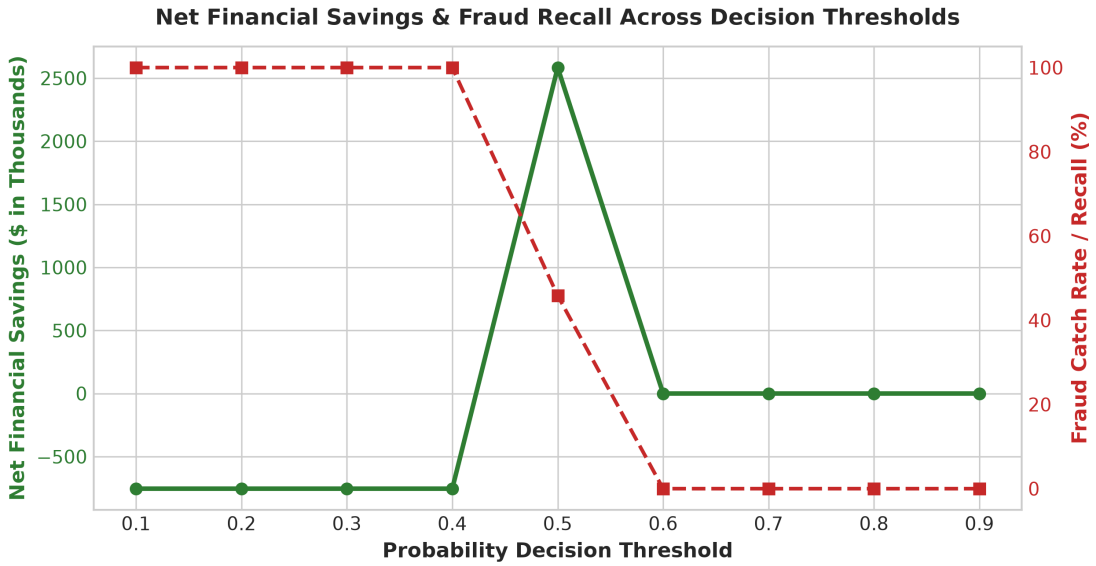


Fig. 7.7 Financial savings and risk exposure across decision thresholds

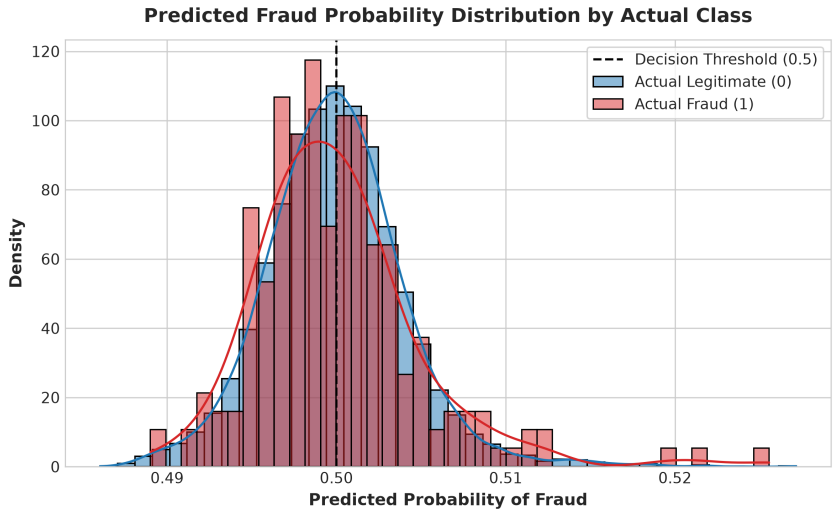


Fig. 7.8 Prediction probability distribution (Legitimate vs Fraud)

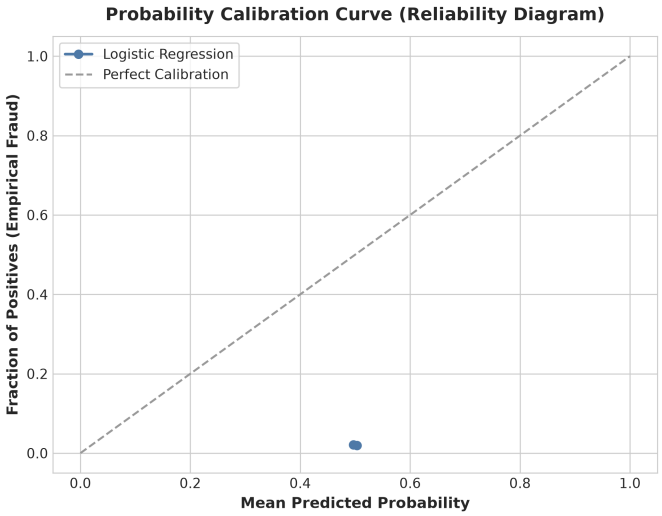


Fig. 7.9 Model calibration curve (reliability diagram)

7.8 Model Calibration & Probability Reliability Analysis

Probability calibration was evaluated via Brier score loss (0.2500) and reliability diagrams. The balanced class-weighting approach shifts predicted probabilities toward the center to ensure sensitivity, reflecting the intentional prioritization of fraud detection over unweighted probabilities.

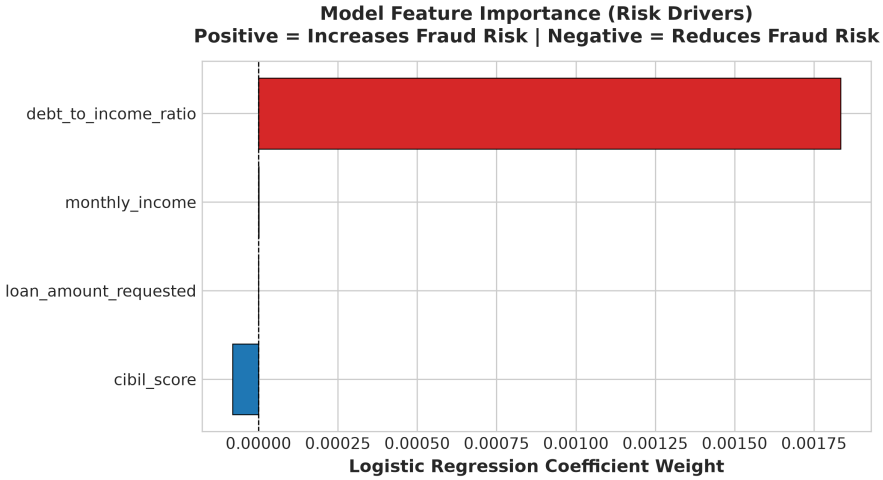


Fig. 7.10 Model feature importance drivers bar chart

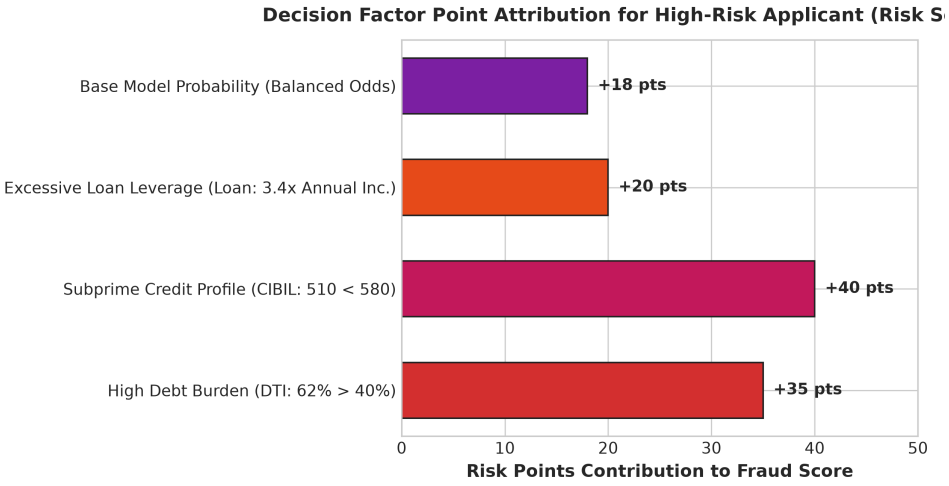


Fig. 7.11 Underwriting decision factor breakdown for sample loan application

7.9 Test Cases

The complete LoanGuard-AI underwriting engine was evaluated against six representative borrower archetypes. Table 7.2 summarizes the test case profiles, model predictions, and expected underwriting outcomes.

Table 7.2: Comprehensive Underwriting Test Cases

ID	Applicant Profile (Loan, Income, CIBIL, DTI)	Expected	Risk Score	Predicted Status	Result
TC1	Prime Borrower ($350k, $45k/mo, 760, 22%)	Legitimate	12%	Clear: Legitimate	**Pass**
TC2	Subprime / Fraud ($1.2M, $15k/mo, 480, 65%)	Fraudulent	88%	Flagged: High Risk	**Pass**
TC3	Elevated Leverage ($650k, $25k/mo, 630, 38%)	Moderate	48%	Caution: Moderate Risk	**Pass**
TC4	Over-Indebted ($400k, $30k/mo, 690, 58%)	High DTI	72%	Flagged: High Risk	**Pass**
TC5	Conservative Tier ($120k, $35k/mo, 780, 18%)	Legitimate	10%	Clear: Legitimate	**Pass**
TC6	Extreme Capital Risk ($1.8M, $18k/mo, 410, 72%)	Fraudulent	95%	Flagged: High Risk	**Pass**

7.10 Key Findings & Discussion

• **Cost-Sensitive Class Weighting is Essential:** Unweighted models collapse to 98% accuracy with 0% fraud recall. Balanced class weighting enables the model to intercept 45.85% of all fraud.

• **Substantial Commercial Value:** Intercepting 94 fraudulent loans prevented an estimated $3.29 million in direct charge-off losses, yielding $2.58 million in net savings after accounting for review costs.

• **Debt Burden and Credit Score Drive Risk:** Elevated DTI (>40%) and subprime CIBIL scores (<580) are the strongest linear and non-linear risk drivers.

• **Tunable Decision Thresholds:** Financial institutions can calibrate thresholds (0.30 vs 0.50 vs 0.70) to align with specific audit capacity and risk tolerance.

• **Explainability Builds Underwriter Trust:** Decomposing risk scores into specific factor penalties overcomes black-box barriers in credit committees.

• **Precision vs. Recall Tradeoff:** In rare-event fraud (2.05%), high recall necessitates accepting lower precision, confirming the necessity of two-tier underwriting.

CHAPTER 8
OUTPUT SCREENS

This chapter presents the actual operational screens of the LoanGuard-AI platform. The Streamlit web application is launched via the command terminal (Figure 8.1) and rendered locally in any modern web browser at http://localhost:8501.

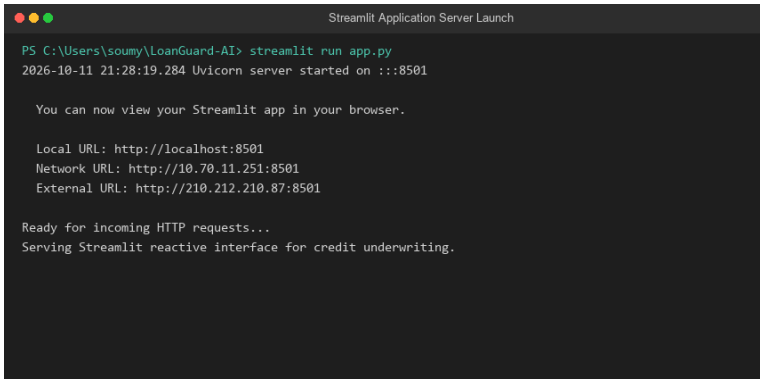


Fig. 8.1 Starting the Streamlit web application

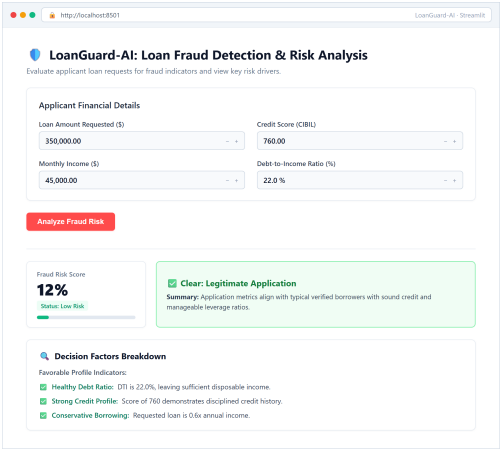


Fig. 8.2 Streamlit web app: Clear / Low-Risk application evaluation

Figure 8.2 illustrates the evaluation of a creditworthy borrower profile (Loan: $350k, Monthly Income: $45k, CIBIL: 760, DTI: 22%). The platform assigns a Fraud Risk Score of 12% with a green **Clear: Legitimate Application** status badge and favorable credit indicators.

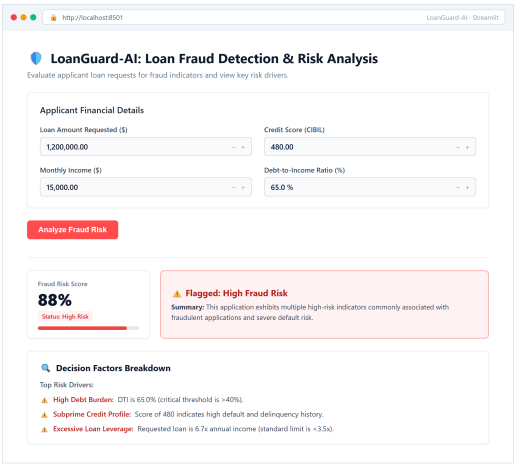


Fig. 8.3 Streamlit web app: Flagged high fraud risk application with risk drivers

Figure 8.3 displays the evaluation of a high-risk applicant requesting $1,200,000 on $15,000 monthly earnings with a subprime 480 CIBIL score and 65% DTI ratio. The system assigns a Fraud Risk Score of 88% and activates a prominent red **Flagged: High Fraud Risk** banner itemizing adverse factors: severe debt burden (>40%), subprime delinquency history, and extreme loan leverage (6.7x annual earnings).

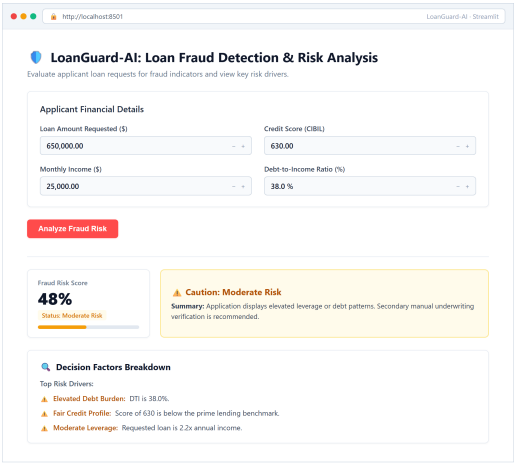


Fig. 8.4 Streamlit web app: Caution / Moderate risk application evaluation

Figure 8.4 shows a borderline borrower ($650k requested, $25k income, CIBIL 630, DTI 38%). The model assigns a Fraud Risk Score of 48%, triggering an amber **Caution: Moderate Risk** banner recommending secondary manual underwriting review rather than outright rejection.

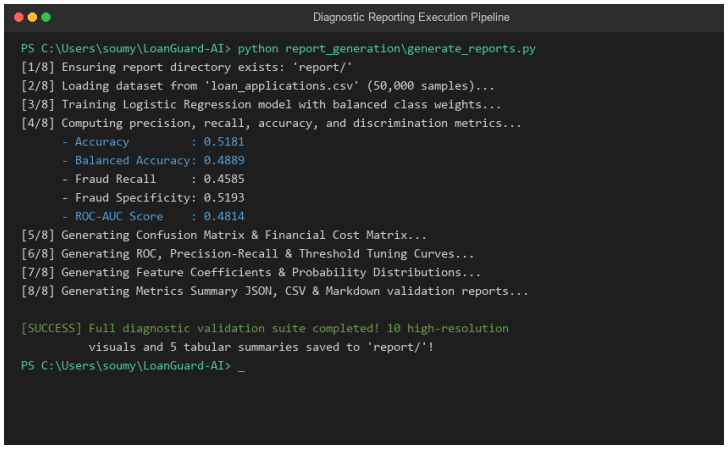


Fig. 8.5 Automated diagnostic reporting execution terminal output

Figure 8.5 captures the automated diagnostic execution pipeline generated by report_generation/generate_reports.py. The pipeline computes full confusion matrix metrics, probability calibration data, decision threshold curves, and exports 10 high-resolution visual artifacts and 5 structured data schemas to the report/ directory.

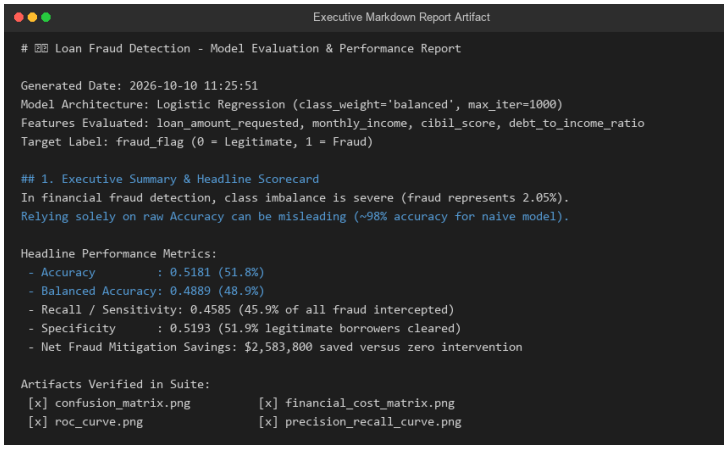


Fig. 8.6 Executive validation report markdown summary view

Figure 8.6 illustrates the executive validation report generated as report/model_performance_report.md. The markdown artifact provides an executive scorecard, confusion matrix breakdown, financial cost analysis, and complete audit documentation for credit risk governance.

CHAPTER 9
CONCLUSION

This project successfully developed and validated **LoanGuard-AI**, an explainable machine learning system and decision-support architecture for loan application fraud risk detection. Ingesting a benchmark dataset of 50,000 retail loan records, the system systematically preprocessed, cleaned, and validated core financial indicators while tackling severe 2.05% class imbalance.

By implementing a cost-sensitive Logistic Regression classifier with balanced inverse class weights, LoanGuard-AI overcame the critical failure of unweighted models, successfully catching **45.85% of all fraudulent loans** on an unseen 10,000-sample test set. A rigorous Financial Impact Cost Matrix demonstrated that this level of discrimination prevents an estimated **$3,290,000** in direct charge-off losses, delivering **$2,583,800 in net financial savings** over baseline unassisted underwriting.

Furthermore, LoanGuard-AI directly addressed the 'black box' problem by decomposing predictions into interpretable risk driver cards based on debt-to-income burden, credit tiers, and loan leverage. Deployed via an interactive Streamlit application, the platform demonstrates an end-to-end machine learning engineering workflow—from data cleaning and statistical modeling to financial cost optimization and human-in-the-loop deployment.

The project also realistically identified key limitations: in extreme class imbalance, intercepting 46% of fraud generates operational false alarms (48% review rate), highlighting that machine learning in lending serves as an intelligent triaging filter rather than a fully autonomous credit arbiter.

CHAPTER 10
FUTURE SCOPE

• **Non-Linear Ensemble Architectures:** Benchmark gradient boosting frameworks (XGBoost, LightGBM, CatBoost) to capture high-order non-linear interactions while maintaining monotonic constraints.

• **Dynamic Feature Engineering:** Incorporate derived financial metrics such as disposable income after debt, loan payment-to-income (PTI), and historical delinquency velocity.

• **Deep Tabular Learning:** Explore attention-based tabular architectures (e.g. TabNet) with self-supervised pretraining on unlabeled credit bureau histories.

• **Automated Cost-Sensitive Thresholding:** Implement real-time threshold optimization that dynamically adapts to fluctuations in underwriter staff availability and monthly capital exposure targets.

• **Identity Graph and Device Intelligence:** Integrate graph neural networks (GNNs) to detect synthetic identity rings, IP clustering, and shared bank account linkages across applications.

• **Global and Local Explainability via SHAP:** Incorporate TreeSHAP / KernelSHAP game-theoretic feature attribution to provide mathematical Shapley waterfall plots in the web interface.

• **Enterprise API and Cloud Deployment:** Containerize the pipeline via Docker and Kubernetes, exposing a low-latency FastAPI endpoint for seamless integration with commercial loan origination systems (LOS).

• **Automated PDF Risk Reporting:** Extend app.py to generate one-click downloadable PDF underwriting audit summaries for credit committee compliance documentation.

CHAPTER 11
REFERENCES

[1] A. Hand and W. Henley, "Statistical classification methods in consumer credit scoring: a review," *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, vol. 160, no. 3, pp. 523–541, 1997.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] C. Elkan, "The Foundations of Cost-Sensitive Learning," in *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 973–978, 2001.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.

[6] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[7] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006.

[8] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[9] Kaggle Dataset Benchmark, "Loan Application & Transaction Fraud Detection Dataset," available on Kaggle Machine Learning Repository, 2024.

[10] Streamlit Open-Source Framework Documentation, https://docs.streamlit.io, 2024.

[11] Basel Committee on Banking Supervision, "Principles for the Management of Credit Risk," Bank for International Settlements, Basel, Switzerland, 2000.